



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Electronics Engineering (SENSE)

**ADAPTIVE MULTI-MODE CONTROL SYSTEM
FOR A ROBOTIC ARM**

(DETAILED REPORT)

Project Presented by: [Group 4]

Aryan Raj Patra (24BLC1222),
Dishan C. Panchigar (24BLC1217),
Neerjyoti Patowary (24BLC1154),
Rudra Purohit (24BLC1152)

1. ABSTRACT

The rapid advancement of modern industrial automation and intelligent robotics brings up a growing need for control mechanisms that are adaptive and user-friendly, reducing to a minimum the need to rely on traditional programming methods. The following project describes a multi-mode robotic arm control system that is designed to handle any robotic arm within its specifications, mechanically and electrically, with intuitive, flexible, and efficient control. This allows users to program in gestures using natural hand movements with no need for complicated coding.

The system consists of two modules: Module-1 is a System Controlling Glove, and Module-2, the Robot Driving System. The modules are connected by Wi-Fi via TCP. The working principle is based on mapping hand orientations and button inputs into the angular motion parameters collectively defined as "Arm-States." These Arm-States represent sets of servo angles corresponding to the base, shoulder, elbow, and claw of the robotic arm. Every gesture or button press would alter one or more of these angles, which would then be transmitted, processed, and executed through the servo driver. A total of up to 30 Arm-States could be stored and replayed sequentially or cyclically for programmable motion sequences. The system allowed for four operating modes. This work demonstrates an idea for a framework for robotic automation.

2. INTRODUCTION:

Automation and robotics have, in the current Industry 4.0 era, become vital pillars of industrial development. With the increased integration of robotic systems into manufacturing, assembly, and precision-based industries, there has been an ever-growing demand for control systems that are efficient, easy to operate, and adaptive. Traditional robotic arms rely on heavy programming interfaces that require technical know-how, hence limiting their operation to skilled personnel. This project will bridge that gap by offering a multi-mode robotic arm control system that enables operation of a robotic arm through various intuitive control modes, such as gesture recognition, pre-programmed motion sequences, and manual joystick override.

The proposed system doesn't require any complex coding through which the operators can control the arm using their natural hand motions. The proposed robot system consists mainly of two modules. Module-1, Robot Controlling Glove, is built using an ESP32 Dev Module embedded with an MPU6050 Inertial Measurement Unit (IMU) that captures the orientation of the hand, and several state buttons for the purpose of claw control, saving, executing, and resetting programs. This information is also displayed on an OLED screen and sent over Wi-Fi via the TCP protocol. Module-2, the Robot Driving System, implemented on Arduino UNO R4 Wi-Fi, is connected to a PCA9685 Servo Driver that drives servos corresponding to the joints-base, shoulder, elbow, and claw-of the robotic arm.

It works on the principle that the detected hand motions and button presses are mapped into predefined angular positions, known as Arm-States. Each Arm-State represents a snapshot of the arm's configuration at any given instant in time, which can then be recorded and replayed sequentially or in loops. This design supports four modes of operation: Hand-Controlled, Programmed Single-Execution, Programmed Loop-Execution, and Joystick Override. The system can easily switch between real-time control and automated playback with great efficiency, flexibility, and accessibility in industrial applications of robotics.

3. COMPONENTS USED:

3[A] Module-1: System Control Glove

- ESP-32 30-Pin Development Board
- An insulative Hand-Glove
- MPU-6050 IMU
- SSD1306 OLED DISPLAY
- Push-Buttons
- Connecting wires

3[2] Module-2: Robot Control System

- Arduino UNO R4 Wi-Fi
- Arduino UNO R3
- PCA9685 16-Channel Servo Shield
- Joystick Modules
- 16x2 LCD Display with I2C interface
- LEDs
- 5V fan
- MB102 Breadboard power module
- Jumper cables
- Bread-Board + MB102 power module



3[C] 4-DOF Robotic Arm (for Demonstration Purpose)

- 4-DOF Robotic Arm body-kit
- SG-90 servo motors

3[D] Software Used:

- Arduino IDE
- WOKWI Cloud Simulation Platform

4. METHODOLOGY:

4[A] Conceptualization:

- **Hand movement**, especially from the joint at wrist, can be mapped as rotation in two axes. Since all these movements lead to unique points, they can act as a way to control a system variable within a particular range. This has led to wrist movements being associated to variables that drive a particular part or system. Hence, the system will require an Inertial Measurement Unit (IMU) or a Gyro-Sensor (GS) to track these movements and convert them into a processable data.
- For a 4 Degree-of-Freedom (DOF) Robotic Arm, having four servo motors driving the arm, each of this servo serves at a particular angle, leading to unique positions achieved by the robotic arm. Hence, in association with the project a term **Arm State** is introduced as:
“A Set of angles and states of all the servo/stepper motors at a particular instance of time.”

[For example, if at a particular instance the robotic arm has these servo angles: Base-65 degrees, Shoulder-120 degrees, Elbow-136 degrees, claw-15 degrees, then Arm State for that instance is: [65,120,136,15] (angles mapped in order: Base, Shoulder, Elbow, Claw). Hence, these Arm States can be used to drive robotic arm, and especially can be used to program a robotic arm. An example being defining 'Initial' and 'Target' Arm States, and that robotic arm automates its servos from initial angles mentioned in 'Initial' Arm State to the target angles mentioned in the 'Target' Arm State.]

- For **Programming a robotic arm for a particular action**, an array of Arm States can be formed, where Arm States can be stored in a sequential order. Hence, when executed, these elemental Arm States are fed to robotic arm in chronological order (from first to last), hence, driving the servo motors of the arm such that they were programed to move/navigate from one position to other automatically.

4[B] Prototyping:

Module-1: System Controlling Glove:

An insulating glove is taken as base, making easy to mount the control system. ESP32 30-pin development board is used for its compact design and integrated Wi-Fi feature. MPU6050 is mounted in the center of back of the palm to sense palm movements. Furthermore, four push-buttons labelled as: CLAW, SAVE, RUN, and RESET, were mounted at the tip, sides and base of index finger their easy access.

The connections were done as follows:

Components	Pins of Components	Pins of ESP32 Dev Board
SSD1306 OLED DISPLAY	VCC	3V3
	GND	GND
	SDA	D18
	SCL	D19
MPU6050 IMU	VCC	3V3
	GND	GND
	SDA	D21
	SCL	D22
CLAW (pull-up)	(IN, OUT)	(D26, GND)
SAVE (pull-up)	(IN, OUT)	(D27, GND)
RUN (pull-up)	(IN, OUT)	(D14, GND)
RESET (pull-up)	(IN, OUT)	(D-12, GND)

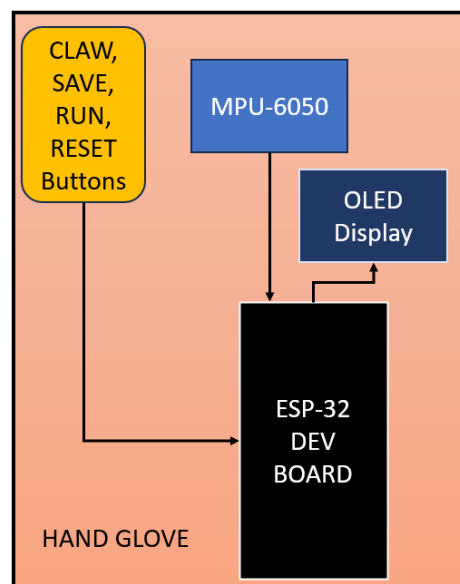


Figure 1.1[a] and Figure 1.1[b] shows the data communications occurring between the components within the module and physical placement of the components used in Module-1.

Furthermore, Figure 1.2 is the circuit diagram for the connections done within Module-1.

Figure 1.1[a] data communications within Module-1

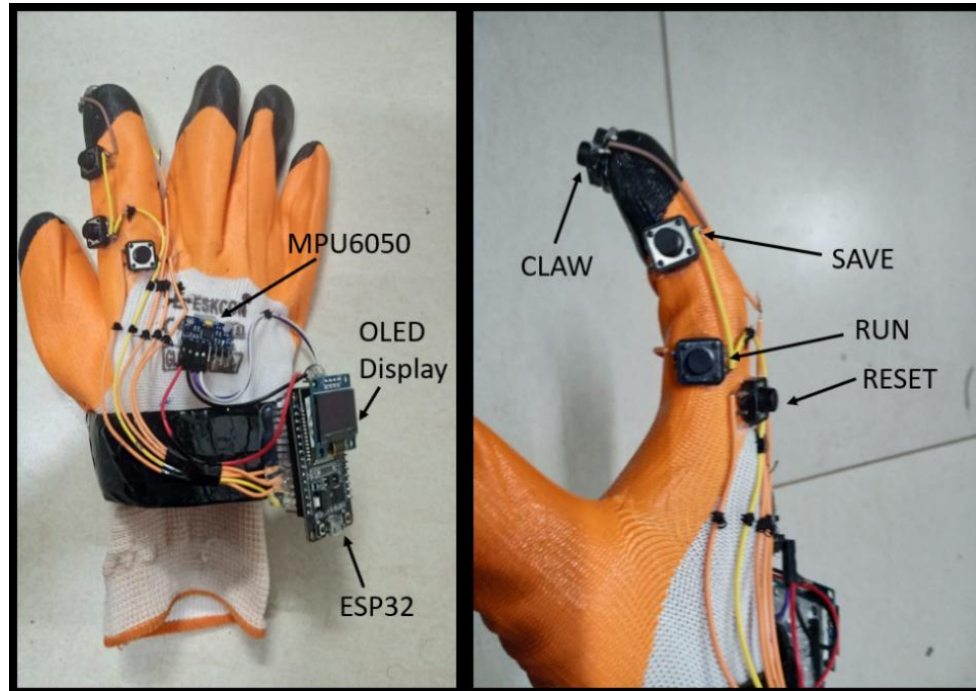


Figure 1.1[b] physical placements of components within module-1

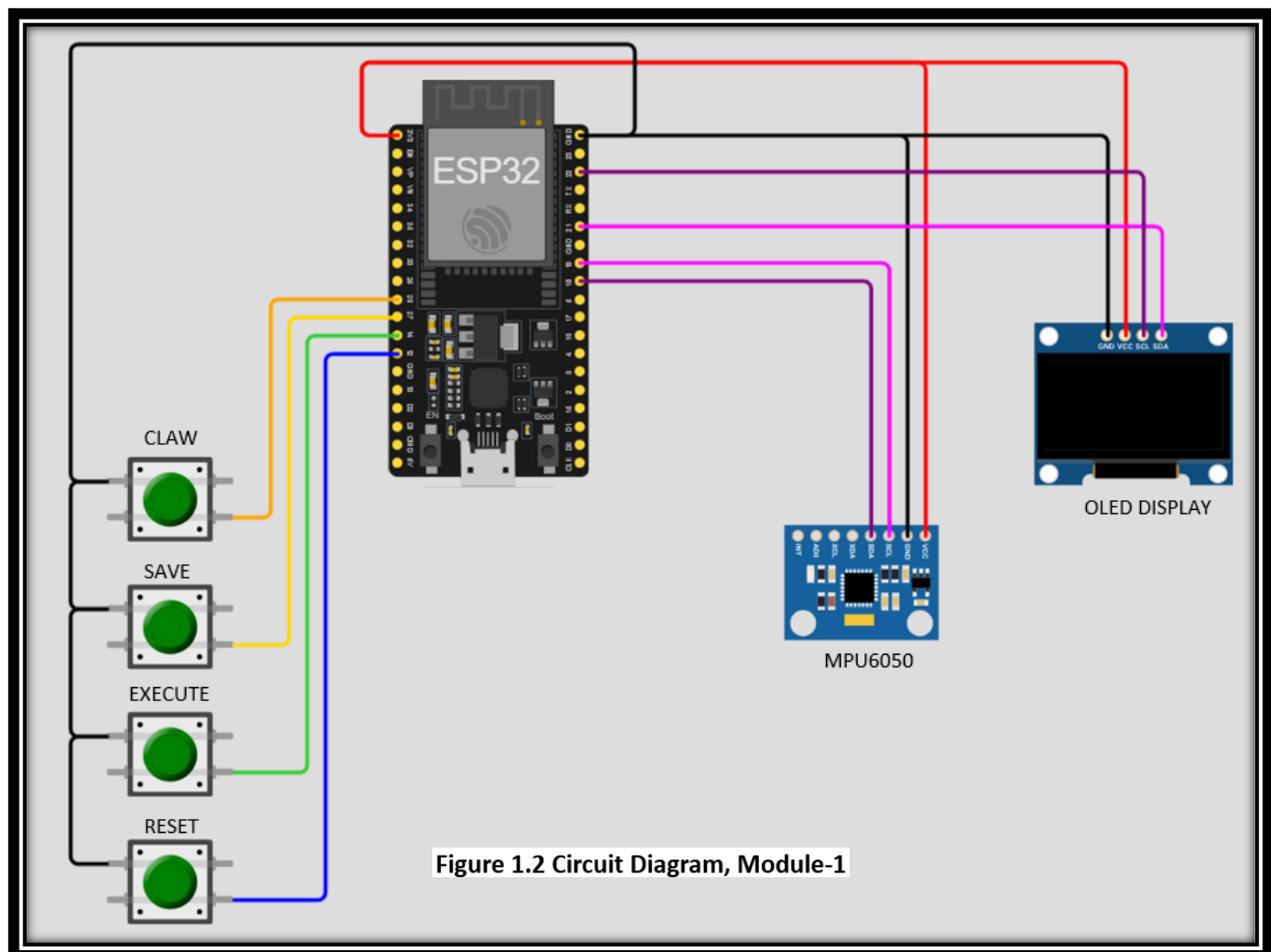


Figure 1.2 Circuit Diagram, Module-1

Module-2: Robot Controlling System:

Arduino R4 Wi-Fi is used as base micro-controller due to its Wi-Fi enabled feature and user-friendliness. Joystick, PCA9685 were connected to the board. Furthermore, servos can be connected to defined channels after the Modules are programmed. The connections in Module-2 were done as given in the Table below:

Components	Pins of Components	Pins of Arduino r4 Wi-Fi Board
Joystick-A	VCC	5V
	GND	GND
	VERT (Vy)	A0
	HORI (Vx)	A1
Joystick-B	VCC	5V
	GND	GND
	VERT (Vy)	A2
	HORI (Vx)	A3
PCA 9685 Servo Shield	VCC	5V
	GND	GND
	V+	5V
	OE	GND
	V+ (Ext)	VCC (external)
	GND	GND (external)
	GND (external)	GND

While Figure 2.1 shows data show how data communication within Module-2. Figure 2.2 shows the circuit diagram of connections within Module-2. After prototyping, more components can be added, like status system and a cooling fan as add-on as shown in Figure 2.3, which was also implemented as the finalized design for Module-2. LEDs were used to indicate connection Status between modules, and LCD display showing override status. A small 5v fan was also used to circulate air within the Module after casing. The accessories were mounted on another Arduino system, using Arduino UNO R3 as shown in the table below. This makes it removable in case of unwanted power distribution or under-performance of the Module. External VCC was provided to breadboard using MB102 module.

Components	Pins of Components	Pins of Arduino UNO R3 Board
LEDs	STATUS LED (+)	D4
	STATUS LED (-)	GND
	OVERRIDE LED (+)	D5
	OVERRIDE LED (-)	GND
Cooling Fan	VCC	VCC (external)
	GND	GND (external)
Arduino UNO R4 Wi-Fi	D3	D2
	D4	D3
	GND	GND
16x2 LCD Display With I2C interface	VCC	5V
	GND	GND
	SDA	A4
	SCL	A5
External Power Supply	VCC (external)	Vin
	GND (external)	GND

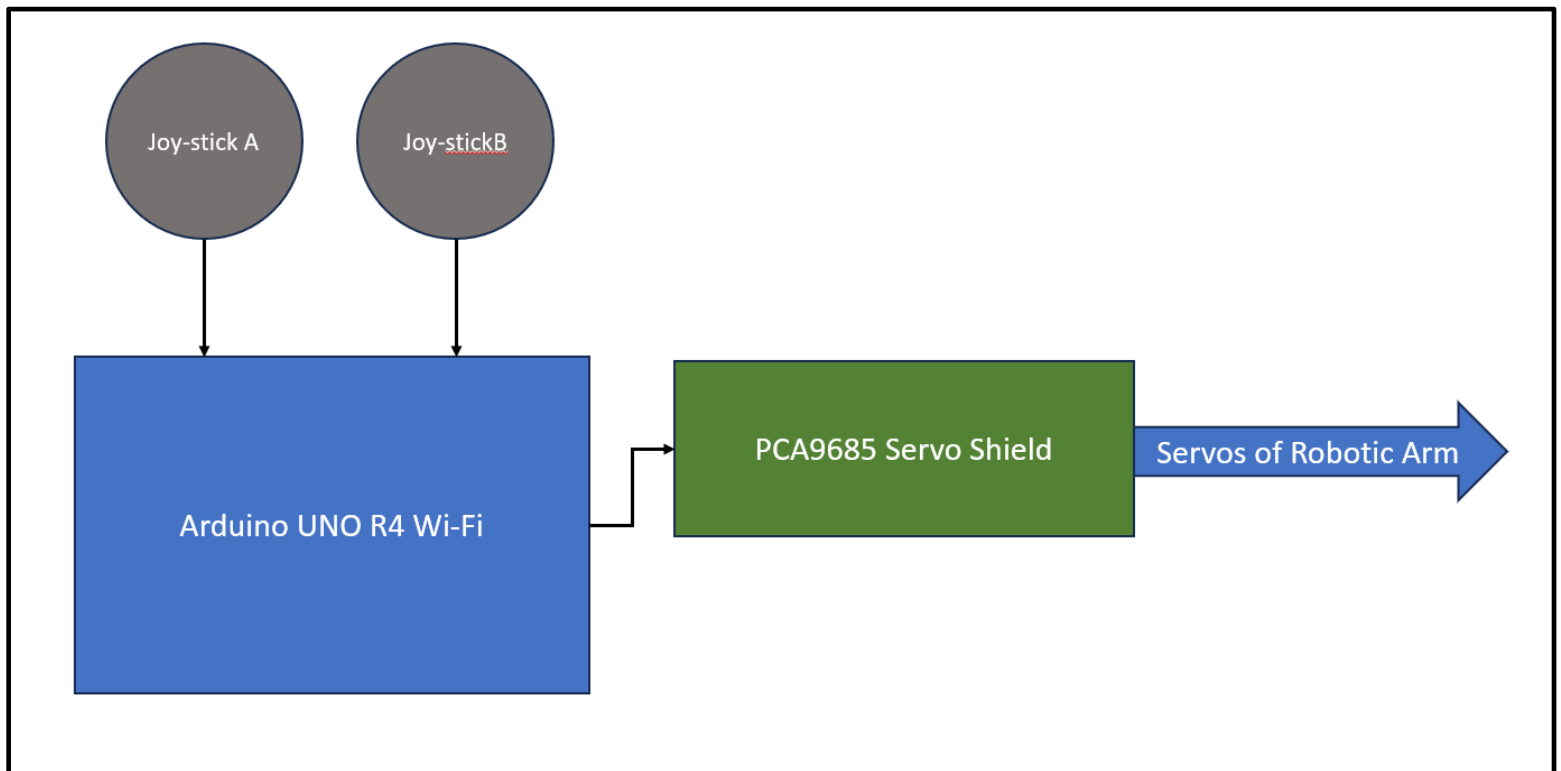


Figure 2.1[a] Data communication within Module-2

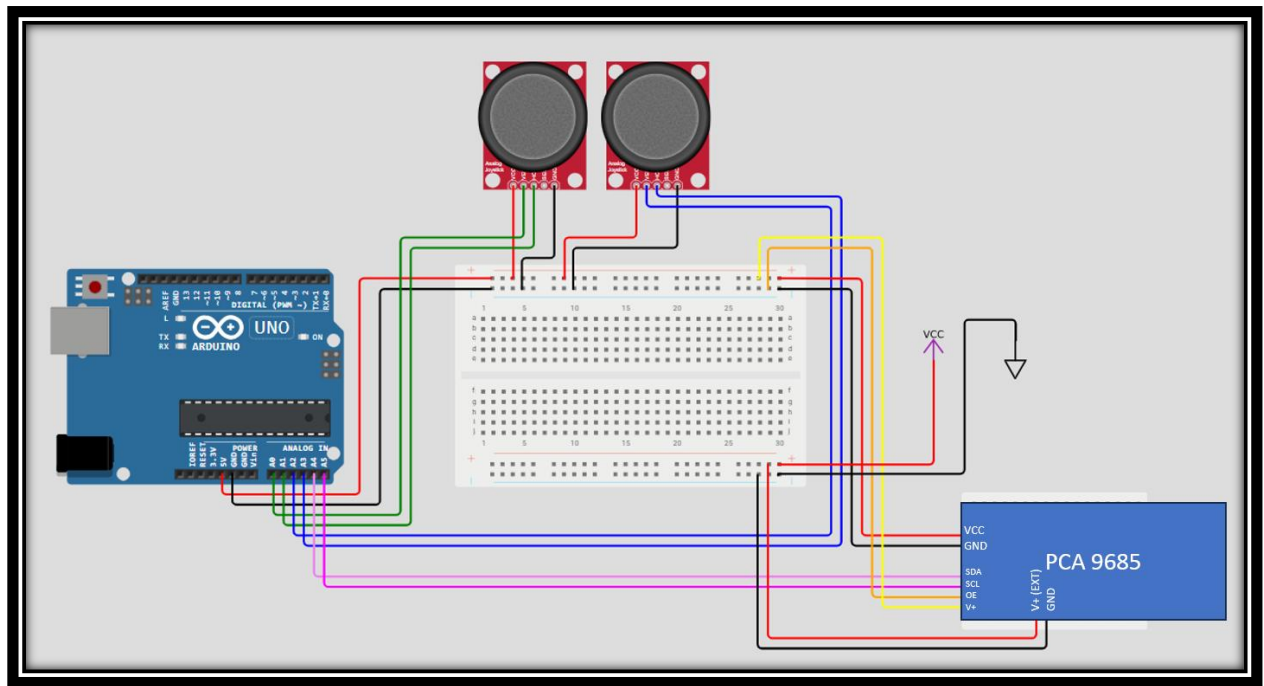


Figure 2.2 circuit Diagram of Module-2 Prototype

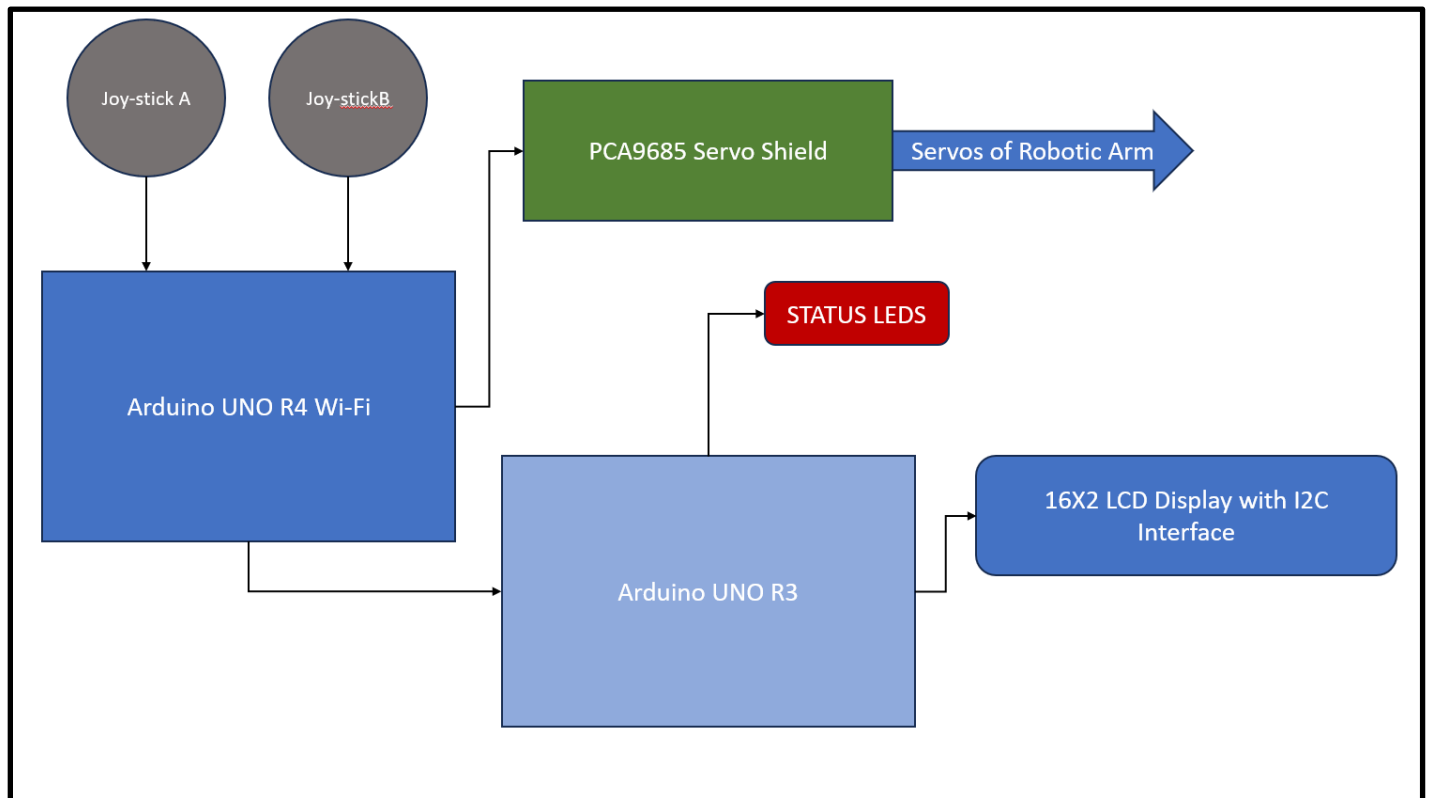


Figure 2.1[b] Data communication within Module-2 with added Accessories

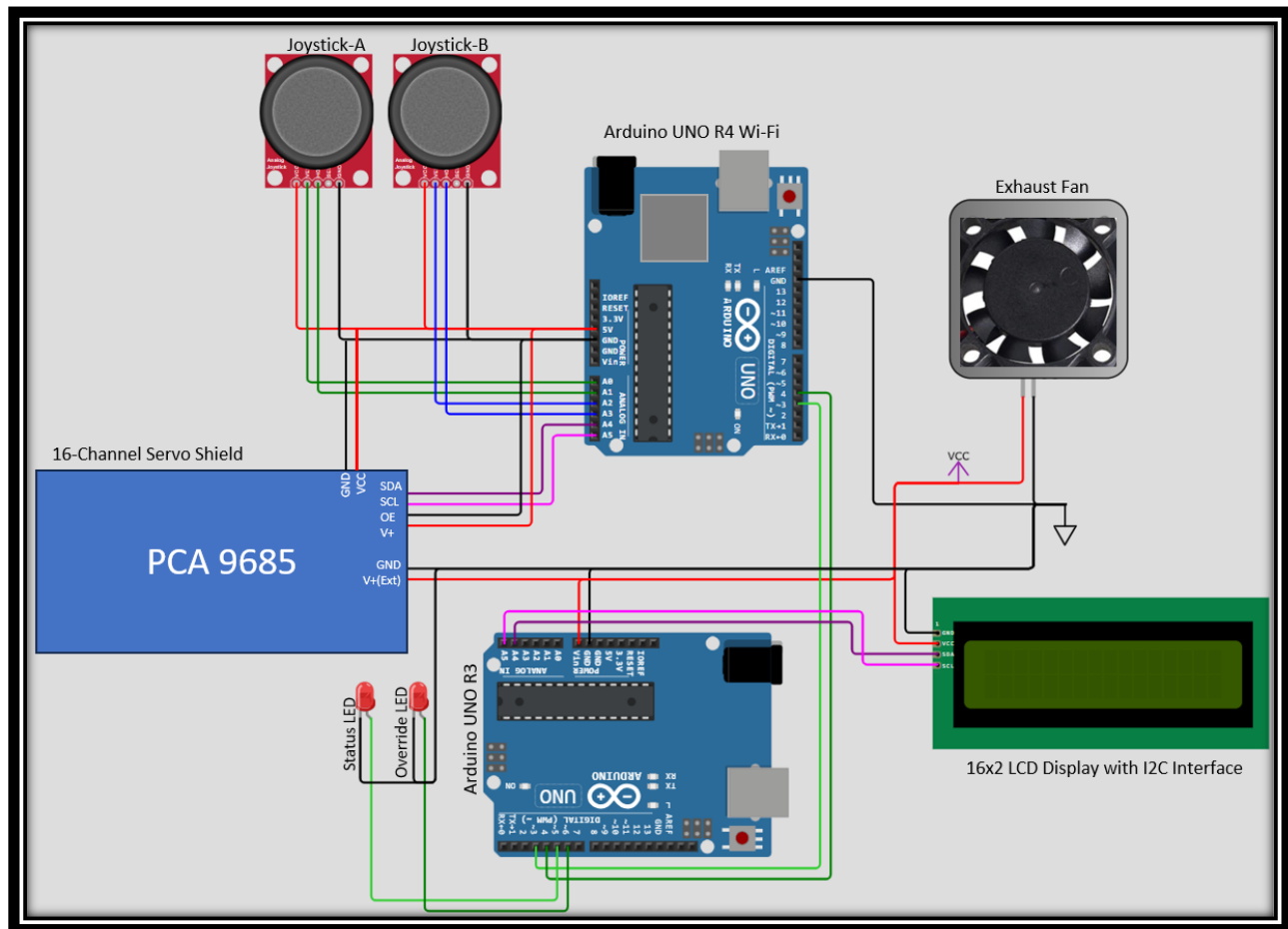


Figure 2.3 Circuit diagram, Module-2 (Added Accessories)

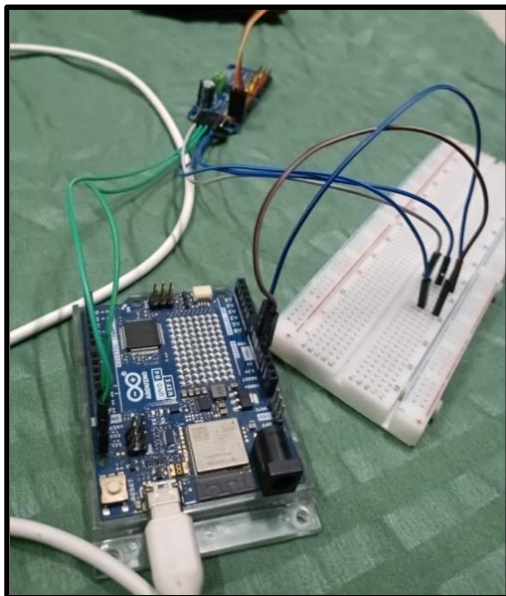


Figure 2.4[a] Module-2 Prototype

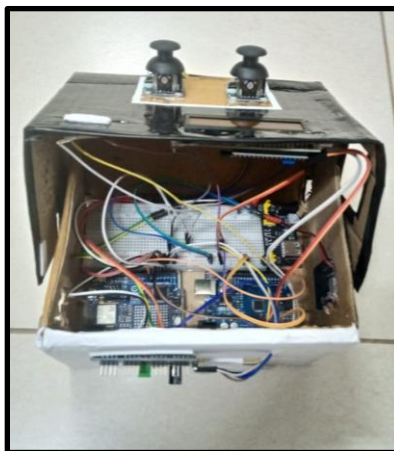


Figure 2.4[b] Module-2 Final Product based on Figure 2.3

Working of Modules:

Module-1 (ESP32) –

It acts as the controller glove module that is responsible for collecting motion and button data and transmitting it wirelessly to the robotic arm. Upon every boot, it checks whether the OLED display and the MPU6050 sensor have been initialized properly. If the response of both the modules is affirmative, it prints “System Ready” on the OLED, indicating that all internal systems have been checked and are functioning. Following this verification, ESP32 activates its onboard Wi-Fi module, attempting to connect to a pre-set network. Once connected, the OLED displays the Wi-Fi name and IP address of the device. Then, ESP32 waits for Module-2 to connect. Once a successful connection is established with Module-2, it commences streaming its key variables continuously to Module-2: Base, Shoulder, Elbow, Claw, Save, Run, and Reset. Furthermore, the display reflects each of the variable states in real-time. The user would always know the current state of operations.

the ESP32 has these variables:

- 1) **base (string)**: 1, if the rotation is in +ve x-axis.
2, if the rotation is in -ve x-axis.
0, if no rotation.
- 2) **Shoulder (string)**: 1, if the rotation is in +ve y-axis.
2, if the rotation is in -ve y-axis.
0, if no rotation.
- 3) **Elbow (string)**: 1, if the rotation is in +ve z-axis.
2, if the rotation is in -ve z-axis.
0, if no rotation.
- 4) **Claw (string)**: 1, if claw button is pressed.
0, if claw button is not pressed.
- 5) **Save (string)**: 1, if save button is pressed.
0, if save button is not pressed.
- 6) **Run (string)**: 1, if run button is pressed.
0, if run button is not pressed.
- 7) **Reset (string)**: 1, if reset button is pressed.
0, if reset button is not pressed.

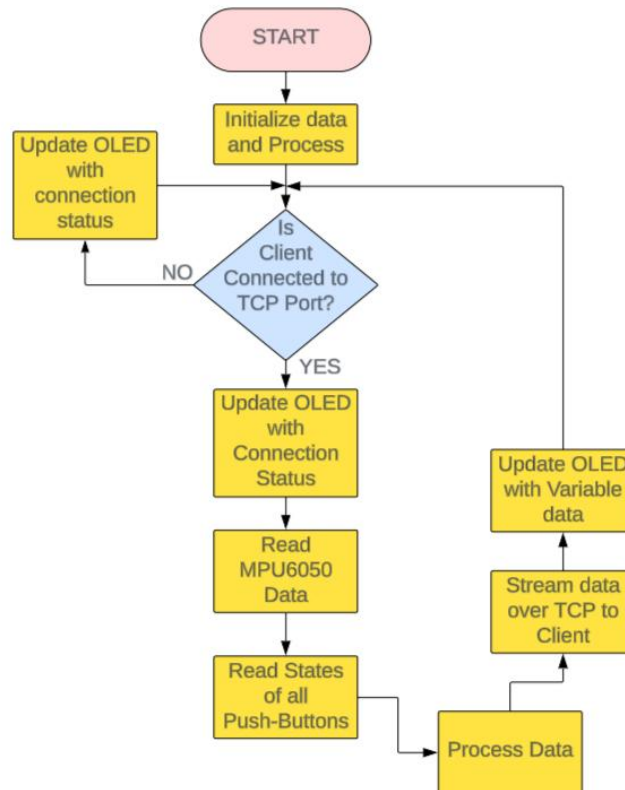


Figure 3.1 Algorithm of Module-1

Module-2 (Arduino UNO R4 Wi-Fi) –

The Arduino UNO R4 Wi-Fi is the receiver and actuator controller, which interprets the commands from the ESP32 into precise servo movements. Initially, it keeps all onboard LEDs blinking to indicate that it is waiting for a connection. As soon as the ESP32 connects, the LEDs stop blinking, and thus it confirms a successful communication. Arduino continuously receives variable data sent from Module-1 and processes it through a structured logic system. Each variable directly controls one of the four servo channels on the PCA9685 driver: Base, Shoulder, Elbow, and Claw. Each servo has a default angle and range. Movements are triggered in 1° increments or decrements according to the received data, respecting constraints such as preventing simultaneous movement between dependent joints like shoulder-elbow. The claw toggles between fully open and fully closed positions. When the Save button is pressed, the current servo states are stored; the pressing of Run replays the stored sequence in order, while Reset clears all saved positions.

During joystick operation, Arduino overrides received Wi-Fi data and directly controls the servos, displaying a dynamic LED pattern that indicates manual override mode.

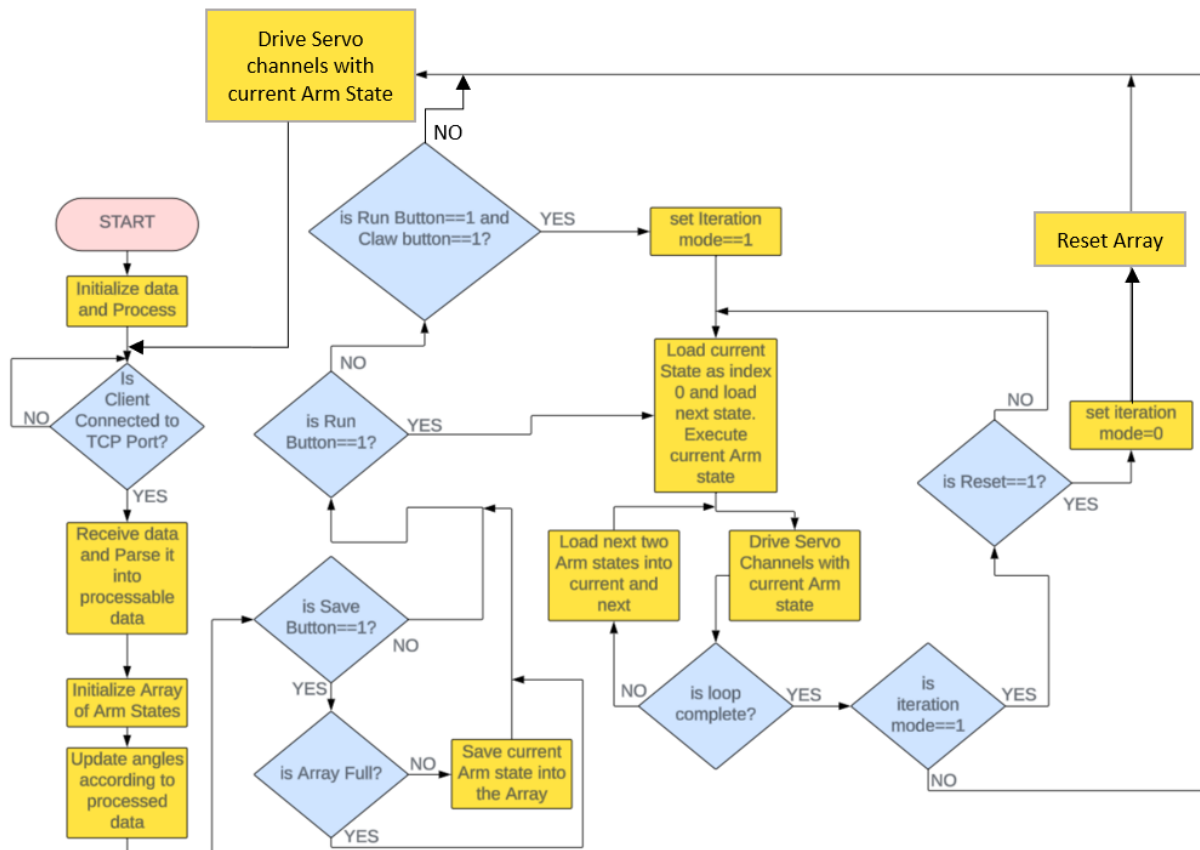


Figure 3.2 Algorithm of Module-2

Arduino is connected to PCA9685 servo driver, which has following associations:

- 1) **Channel-0** is associated to Base angle.
- 2) **Channel-1** is associated to Shoulder angle.
- 3) **channel-2** is associated to Elbow angle.
- 4) **channel-3** is associated to Claw angle.

Details for all the angles are as follows:

Base angle: range: 0-180 degrees, default: 90 degrees

Shoulder angle: range: 120-160 degrees, default: 135 degrees

Elbow angle: range: 0-30 degrees, default: 90 degrees

claw angle: range: 0-90 degrees, default: 90 degrees

Details of how these angles are affected:

1)Base angle should:

increase by 1 degree, when received value of Base=1.

decrease by 1 degree, when received value of Base=2.

remain unchanged, when received value of Base=0.

[Constraints: NONE].

2)Shoulder angle should:

increase by 1 degree, when received value of Shoulder=1.

decrease by 1 degree, when received value of Shoulder=2.

remain unchanged, when received value of Shoulder=0.

[Constraints: should not change when there is a change in Base Angle].

3) Elbow angle should:

increase by 1 degree, when received value of Elbow=1.

decrease by 1 degree, when received value of Elbow=2.

remain unchanged, when received value of Elbow=0.

[Constraints: should not change when there is a change in Shoulder Angle].

4) Claw angle should:

90 degrees, when received value of Claw=1.

0 degree, when received value of Claw=0.

[Constraints: NONE].

Details on How Robotic arm will be Programmed and executed with Auto Action:

Arm State, which indicates angles of all servo motors in the Robotic Arm, can be used to program it by storing several of their instances. Hence, for programming Actions, an Array of Size 30 has been initialized.

- 1) When SAVE variable is equal to 1: the current Arm state is saved to the array
- 2) When RUN variable is equal to 1: all the Arm States stored in Array are driven one-by-one until end Arm State is Reached.
- 3) When RUN and CLAW both are equal to 1: the Arm States stored in the Array are driven in a continuous loop until Reset equals to 1.

- 4) When RESET variable is equal to 1: all the Arm States stored in the Array will be flushed out. Hence, refreshing the array for reuse.

Details on Operating modes:

- 1) **Hand-Controlled:** in this mode, the Robotic Arm can be controlled directly using wrist movements. This is the default mode of operating the Arm.
- 2) **Programmed Single-Execution:** in this mode, operator must first save The Arm States, i.e. positions of the robotic arm using SAVE button, and then press on RUN button to execute the saved sequence of Arm States for once.
- 3) **Programmed Loop-Execution:** in this mode, operator must first save The Arm States, i.e. positions of the robotic arm using SAVE button, and then press on RUN + CLAW button to execute the saved sequence of Arm States in a continuous loop. To exit this mode, press RESET button.
- 4) **Joystick Override:** this mode is activated when operator operates joystick. The stretch of joystick is directly mapped with the servo angles and hence, drives them immediately. This mode has the most priority regardless of the robotic arm in any of the above three modes.

5. Results:

After an extensive period of prototyping and testing, the following outcomes were achieved:

1. Seamless Communication:

Non-lagging and stable communication was successfully established between both modules, ensuring real-time data transfer and responsiveness.

2. Accurate Gesture Response:

The system accurately responded to various hand gesture inputs, correctly translating them into the corresponding robotic arm movements.

3. Reliable Mode Switching:

Smooth and error-free transitions were achieved while switching between different operational modes, maintaining consistent control performance.

4. **Successful Arm State Management:**

The system was able to save multiple arm states effectively. It also retrieved and executed these states flawlessly in both Programmed Single-Execution and **Programmed Loop-Execution** modes.

The arm movements during state transitions were observed to be stable and smooth, without any jerks or loss of synchronization.

5. **Effective Override Functionality:**

The Joystick Override mode successfully took priority over all other operational modes, allowing manual real-time control of the robotic arm whenever required.

6. **Conclusion:**

The Wi-Fi–based gesture-controlled robotic arm successfully achieved smooth, real-time communication between the ESP32 controller glove and Arduino UNO R4 Wi-Fi. The system accurately responded to motion and button inputs, executing precise servo movements through the PCA9685 driver. Multiple operating modes—Hand-Controlled, Programmed Execution, and Joystick Override—were implemented effectively, with stable data transfer and minimal latency. The LED matrix visualization further enhanced user interaction.

Future Scope:

The system can be upgraded with cloud-based data logging, camera-assisted feedback for visual guidance, and AI-driven motion prediction for enhanced precision. Integration with mobile or web dashboards could enable remote monitoring and control.

7. Gallery:

Prototyping and testing Stage:

